

Smart Grid Decision Load Agent

GROUP-20

2520090174-RISHIL

2520090177-KEERTHI

2520090178-AKSHITHA

Abstract

- ▶ This project presents a **Smart Grid Load Decision Agent** that allocates available electrical power to different demand sectors (e.g., industrial, residential, hospital) based on priority levels. The agent implements a deterministic, rule-based decision strategy that sorts sectors by priority (highest first) and allocates power up to their requested demand, respecting the total available capacity. A complete trace log records each allocation step, enabling full explainability. The system is designed to handle emergency scenarios where available power is insufficient, ensuring critical sectors receive power first.

Introduction

- ▶ Mobile robot localization is the task of determining a robot's exact position within a known environment while avoiding obstacles and reaching a goal. This project implements a hybrid AI system that combines multiple techniques to solve this problem in a 5×5 grid world, where the robot starts at (0,0), the goal is at (4,4), and six obstacles block certain cells. The Python code is organized into six modules aligned with course outcomes CO1 to CO6. CO1 models the environment using a 2D grid with symbols S (start), G (goal), X (obstacle), and 0 (free space). CO2 applies breadth-first search (BFS) with a queue and visited set to find the shortest path from start to goal. CO3 adds constraint satisfaction by checking that every step avoids obstacle cells, rejecting invalid moves. CO4 introduces utility-based decision making, where each path is scored using the formula $\text{Utility} = 100 - (\text{Distance} \times \text{Risk})$, allowing the robot to choose the safest route among equally short paths. CO5 implements Bayesian localization: given a prior probability (0.6) and a sensor reading at (2,2) with 80% accuracy, the posterior probability is calculated using Bayes' rule, yielding 0.686 to estimate the robot's true position. Finally, CO6 integrates all previous techniques—BFS search, constraint validation, Bayesian belief update, and utility evaluation—into a single hybrid reasoning pipeline.

Goals

- ▶ **Primary Goal:** Maximise the supply of power to high-priority sectors under a limited total power budget.
- ▶ **Secondary Goal:** Provide a transparent, step-by-step reasoning trace for each allocation decision.
- ▶ **Constraint Satisfaction:** Never allocate more power than the available total.
- ▶ **Fairness & Efficiency:** Avoid waste by allocating exactly the requested demand whenever possible, and use remaining power fairly among lower-priority sectors.

Problem Statement

- ▶ In a smart grid environment, the central controller must decide how to distribute limited electrical power among multiple demand sectors (e.g., hospital, industrial, residential). Each sector has:
- ▶ A **power demand** (kW) that it would ideally consume.
- ▶ A **priority** (1 = lowest, 5 = highest) indicating its criticality.
- ▶ The total available power is fixed and may be less than the sum of all demands. The agent must allocate power to sectors in a way that:
- ▶ Never exceeds the total available power.
- ▶ Respects priority (higher priority sectors get their full demand first).
- ▶ If power is insufficient, lower-priority sectors receive whatever remains, possibly less than their demand.
- ▶ Provide a clear, auditable trace of all decisions

System Environment

- ▶ **Agent:** SmartGridAgent (decision-making entity).
- ▶ **Environment:** Static, deterministic, discrete time (single allocation round). The environment consists of a set of sectors, each with known demand and priority, and a single numerical value representing total available power.
- ▶ **Actuators:** The agent sets the allocated attribute for each sector.
- ▶ **Sensors:** The agent reads sector demands, priorities, and the available power value (input by user).
- ▶ **Interface:** Command-line input/output; the user provides available power, number of sectors, and each sector's name, demand, and priority.

▶ **CO1**

- ▶ **Objective:** Model environment, PEAS, problem formulation, Python basics, trace logging
- ▶ **Concepts:** Grid state, obstacle set, BFS queue, dataclasses, step-by-step trace
- ▶ **Output:** 5×5 grid with S/G/X, start (0,0), goal (4,4), obstacles listed

▶ **CO2**

- ▶ **Objective:** Implement BFS to find shortest path
- ▶ **Concepts:** Queue, visited set, graph search on grid
- ▶ **Output:** Shortest path of length 9, goal reached

▶ **CO3**

▶ **Objective:** Validate path constraints (no obstacles)

▶ **Concepts:** Constraint satisfaction, forward checking, failure explanation

▶ **Output:** All steps ✓, no violations, valid path confirmed

▶ **CO4**

▶ **Objective:** Choose best path using utility function

▶ **Concepts:** $Utility = 100 - (Distance \times Risk)$, policy selection

▶ **Output:** Path B (risk=2, utility=89) selected over Path A (risk=8, utility=83)

▶ **CO5**

▶ **Objective:** Estimate robot position using Bayes' rule

▶ **Concepts:** Prior=0.6, sensor accuracy=0.8, posterior probability

▶ **Output:** Estimated position (2,2), posterior belief = 0.686

▶ **CO6**

▶ **Objective:** Integrate search + CSP + Bayes + utility into hybrid AI

▶ **Concepts:** Pipeline, explainable traces, performance & limitations

▶ **Output:** Path found, constraints satisfied, belief=0.686, utility=91, goal reached

Performance Analysis

CO	Main Technique	Output
CO1	Environment Modeling (PEAS, grid state)	5×5 grid with S, G, X; start/goal defined
CO2	BFS Search (queue + visited set)	Shortest path of length 9 from (0,0) to (4,4)
CO3	Constraint Satisfaction (obstacle validation)	All path steps valid, no constraint violations
CO4	Utility Function ($100 - \text{Distance} \times \text{Risk}$)	Path B selected (Utility=89) over Path A (83)
CO5	Bayesian Reasoning (Bayes' rule)	Estimated position (2,2), posterior belief = 0.686
CO6	Hybrid AI (BFS + CSP + Bayes + Utility)	Goal reached, constraints satisfied, utility=91

Observations

- ▶ **Priority Sorting Works:** The agent correctly sorts sectors by descending priority before allocation. For example, a hospital (priority 5) will be served before residential (priority 3).
- ▶ **Constraint Handling:** When remaining power is less than a sector's demand, the agent activates the constraint, allocates the remaining power to that sector, and stops further allocation.
- ▶ **Trace Clarity:** Each allocation step is logged, including when a constraint is triggered. This makes the decision process fully explainable.
- ▶ **Edge Cases:**
 - ▶ If two sectors have the same priority, the one listed first (after sorting) receives power first – this can be improved by tie-breaking rules.
 - ▶ If available power is zero, all sectors receive zero allocation.
- ▶ **Limitation:** The agent does not consider future time steps or variable renewable generation; it is a one-shot allocator.

► Co-1 Output:

```
Sector: Hospital  
Demand: 80  
Priority: 5  
Decision: High Load
```

► CO-2 Output

```
Greedy Search Result  
  
Hospital 5  
Mall 4  
School 3
```

```
Enter Number of Sectors: 3
```

```
Sector 1
```

```
Name: Hospital
```

```
Demand: 80
```

```
Priority: 5
```

```
Sector 2
```

```
Name: School
```

```
Demand: 40
```

```
Priority: 3
```

```
Sector 3
```

```
Name: Mall
```

```
Demand: 60
```

```
Priority: 4
```

► Co-3 Output:

```
Available Power: 100  
Number of Sectors: 3  
Sector Name: Hospital  
Demand: 80  
Sector Name: School  
Demand: 50  
Sector Name: Mall  
Demand: 30
```

```
Constraint Failed for School  
Hospital Allocated: 80  
School Allocated: 20  
Mall Allocated: 0
```


► CO-4 Output:

```
Number of Sectors: 2  
Name: Hospital  
Demand: 80  
Priority: 5  
Name: School  
Demand: 40  
Priority: 3
```

```
Utility = 400  
Utility = 120
```

► CO-5 Output:

```
Overload Probability = 0.4
```

► CO-6 Output:

```
Hospital | Utility = 400 | Risk = 0.8 | Decision = High Risk  
Mall | Utility = 360 | Risk = 0.9 | Decision = High Risk  
School | Utility = 120 | Risk = 0.4 | Decision = Normal Allocation
```

```
Number of Sectors: 3
```

```
Sector 1
```

```
Name: Hospital
```

```
Demand: 80
```

```
Priority: 5
```

```
Sector 2
```

```
Name: School
```

```
Demand: 40
```

```
Priority: 3
```

```
Sector 3
```

```
Name: Mall
```

```
Demand: 90
```

```
Priority: 4
```

Result

- ▶ The hybrid AI system successfully achieved all objectives. The BFS search algorithm found the shortest path from start (0,0) to goal (4,4) with a length of 9 steps. Constraint satisfaction validation confirmed that every step in the path avoided obstacle cells, with no constraint violations detected. Bayesian localization estimated the robot's position at (2,2) with a posterior belief of 0.686, given a prior probability of 0.6 and sensor accuracy of 0.8. The utility-based decision maker compared two alternative paths and selected Path B because it delivered a higher utility score (89) compared to Path A (83), primarily due to its lower risk value. Finally, the integrated hybrid AI pipeline combined BFS search, constraint checking, Bayesian localization, and utility evaluation to produce the final optimal path: $S \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (3,4) \rightarrow G$. The goal was reached successfully, localization was updated accurately, and the hybrid AI system executed without errors.

Conclusion

- ▶ The Smart Grid Load Decision Agent successfully implements a priority-based power allocation strategy that is simple, efficient, and transparent. It satisfies all stated goals and constraints. The trace log provides valuable insight for system operators and can be extended to include more sophisticated reasoning, such as probabilistic demand forecasting, search-based optimisation (UCS/A*), or constraint satisfaction (CSP) for multi-period scheduling. The current version serves as a solid baseline for more advanced smart grid decision agents that incorporate uncertainty, learning, and multi-agent coordination.



▶ Thank You